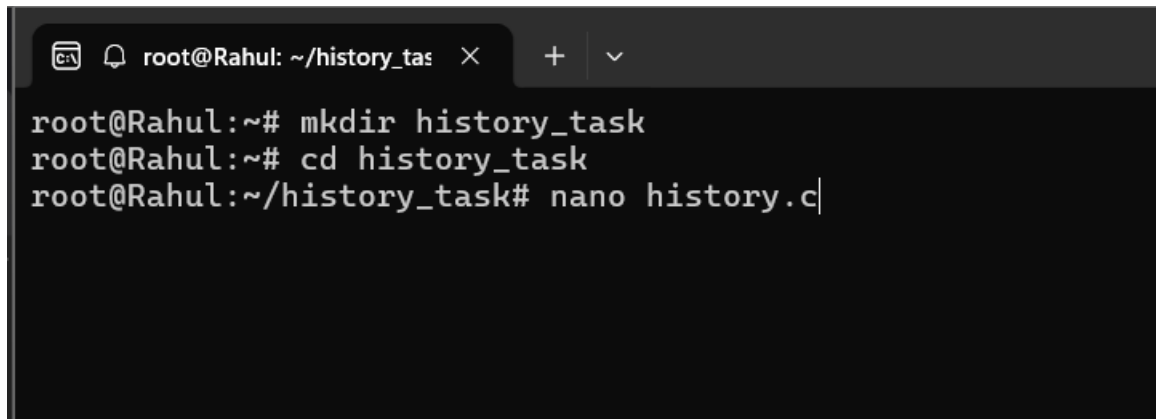


Skill - 6

Task -1:

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency



```
root@Rahul: ~/history_tas × + v
root@Rahul:~# mkdir history_task
root@Rahul:~# cd history_task
root@Rahul:~/history_task# nano history.c|
```

Code:

```
#include <stdio.h>
#include <string.h>
#define MAX_HISTORY 5
#define MAX_CMD 100
char history[MAX_HISTORY][MAX_CMD];
int count = 0;
void add_command(char *cmd) {
    if (count < MAX_HISTORY) {
        strcpy(history[count], cmd);
        count++;
    } else {
        for (int i = 0; i < MAX_HISTORY - 1; i++) {
            strcpy(history[i], history[i + 1]);
        }
    }
}
```

```

    strcpy(history[MAX_HISTORY - 1], cmd);
}
}

void display_history() {
    printf("\nCommand History:\n");
    if (count == 0) {
        printf("History is empty.\n");
        return;
    }
    for (int i = 0; i < count; i++) {
        printf("%d %s\n", i + 1, history[i]);
    }
}

void retrieve_command(int position) {
    if (position < 1 || position > count) {
        printf("Invalid history position.\n");
    } else {
        printf("Command %d: %s\n", position, history[position - 1]);
    }
}

int main() {
    char command[MAX_CMD];
    int choice, position;
    while (1) {
        printf("\n1. Store Command");
        printf("\n2. Display History");
        printf("\n3. Retrieve Command");
        printf("\n4. Exit");
    }
}

```

```
printf("\nEnter choice: ");  
scanf("%d", &choice);  
getchar();  
if (choice == 1) {  
    printf("Enter command: ");  
    fgets(command, MAX_CMD, stdin);  
    command[strcspn(command, "\n")] = '\0';  
    add_command(command);  
    printf("Command stored successfully.\n");  
}  
else if (choice == 2) {  
    display_history();  
}  
else if (choice == 3) {  
    printf("Enter history position: ");  
    scanf("%d", &position);  
    retrieve_command(position);  
}  
else if (choice == 4) {  
    printf("Exiting...\n");  
    break;  
}  
else {  
    printf("Invalid choice.\n");  
}  
}  
return 0;  
}
```

Output:

```
root@Rahul: ~/history_task × + ∨

root@Rahul:~# mkdir history_task
root@Rahul:~# cd history_task
root@Rahul:~/history_task# nano history.c
root@Rahul:~/history_task# gcc history.c -o history
root@Rahul:~/history_task# ./history

1. Store Command
2. Display History
3. Retrieve Command
4. Exit
Enter choice: 1
Enter command: ls
Command stored successfully.

1. Store Command
2. Display History
3. Retrieve Command
4. Exit
Enter choice: 1
Enter command: pwd
Command stored successfully.

1. Store Command
2. Display History
3. Retrieve Command
4. Exit
Enter choice: 1
Enter command: cd Documents
Command stored successfully.

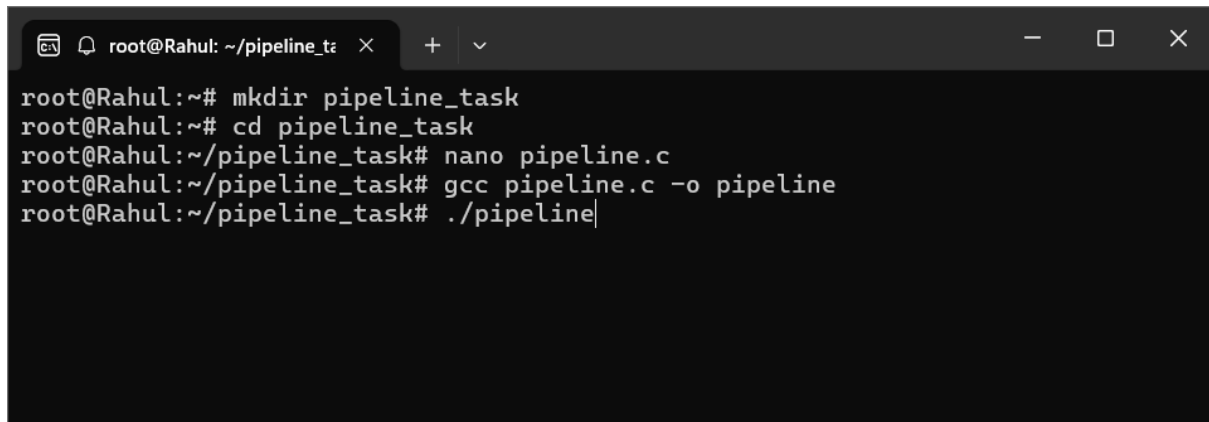
1. Store Command
2. Display History
3. Retrieve Command
4. Exit
Enter choice: 2

Command History:
1  ls
2  pwd
3  cd Documents

1. Store Command
2. Display History
3. Retrieve Command
4. Exit
Enter choice: 3
Enter history position: 2
Command 2: pwd
```

Task -2:

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

A terminal window with a dark background and light text. The window title bar shows 'root@Rahul: ~/pipeline_t...' and standard window controls. The terminal content shows a series of commands: 'mkdir pipeline_task', 'cd pipeline_task', 'nano pipeline.c', 'gcc pipeline.c -o pipeline', and './pipeline'.

```
root@Rahul:~# mkdir pipeline_task
root@Rahul:~# cd pipeline_task
root@Rahul:~/pipeline_task# nano pipeline.c
root@Rahul:~/pipeline_task# gcc pipeline.c -o pipeline
root@Rahul:~/pipeline_task# ./pipeline|
```

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/wait.h>

#define MAX_COMMANDS 5

#define MAX_ARGS 10

#define MAX_LENGTH 100

typedef struct {

    char command[MAX_LENGTH];

    char *args[MAX_ARGS];

} Command;

void parse_command(char *input, Command *cmd) {

    int i = 0;

    cmd->args[i] = strtok(input, " \n");

    while (cmd->args[i] != NULL && i < MAX_ARGS - 1) {

        i++;

        cmd->args[i] = strtok(NULL, " \n");
```

```

    }

    if (i == 0) {

        cmd->command[0] = '\0';

        return;

    }

    strcpy(cmd->command, cmd->args[0]);
}

void execute_pipeline(Command commands[], int count) {

    int pipes[MAX_COMMANDS - 1][2];

    pid_t pids[MAX_COMMANDS];

    for (int i = 0; i < count - 1; i++) {

        if (pipe(pipes[i]) == -1) {

            perror("pipe");

            exit(1);

        }

    }

    for (int i = 0; i < count; i++) {

        pids[i] = fork();

        if (pids[i] == -1) {

            perror("fork");

            exit(1);

        }

        if (pids[i] == 0) {

            if (i > 0) {

                dup2(pipes[i - 1][0], STDIN_FILENO);

            }

            if (i < count - 1) {

                dup2(pipes[i][1], STDOUT_FILENO);

```

```

    }

    for (int j = 0; j < count - 1; j++) {

        close(pipes[j][0]);

        close(pipes[j][1]);

    }

    execvp(commands[i].args[0], commands[i].args);

    perror("execvp");

    exit(1);

}

}

for (int i = 0; i < count - 1; i++) {

    close(pipes[i][0]);

    close(pipes[i][1]);

}

for (int i = 0; i < count; i++) {

    waitpid(pids[i], NULL, 0);

}

}

int main() {

    Command commands[MAX_COMMANDS];

    char input[MAX_LENGTH];

    char *parts[MAX_COMMANDS];

    int count = 0;

    printf("Enter pipeline commands separated by | \n");

    printf("Example: ls | grep .c | wc -l\n");

    printf("Enter pipeline: ");

    fgets(input, sizeof(input), stdin);

    char *token = strtok(input, "|");

```

```
while (token != NULL && count < MAX_COMMANDS) {  
    parts[count] = token;  
    count++;  
    token = strtok(NULL, "|");  
}  
if (count == 0) {  
    printf("Invalid pipeline.\n");  
    return 1;  
}  
printf("\nPipeline Layout:\n");  
for (int i = 0; i < count; i++) {  
    while (*parts[i] == ' ')  
        parts[i]++;  
    parse_command(parts[i], &commands[i]);  
    if (commands[i].command[0] == '\0') {  
        printf("Invalid command in pipeline.\n");  
        return 1;  
    }  
    printf("Process %d: %s\n", i + 1, commands[i].command);  
}  
printf("\nExecuting pipeline...\n\n");  
execute_pipeline(commands, count);  
return 0;  
}
```

Output:

```
root@Rahul: ~/pipeline_task × + ∨
root@Rahul:~# mkdir pipeline_task
root@Rahul:~# cd pipeline_task
root@Rahul:~/pipeline_task# nano pipeline.c
root@Rahul:~/pipeline_task# gcc pipeline.c -o pipeline
root@Rahul:~/pipeline_task# ./pipeline
Enter pipeline commands separated by |
Example: ls | grep .c | wc -l
Enter pipeline: ls | wc -l

Pipeline Layout:
Process 1: ls
Process 2: wc

Executing pipeline...

2
root@Rahul:~/pipeline_task# |
```